

SlimSpec: Low-Rank Draft LM-Head for Accelerated Speculative Decoding

Anton Plaksin* Sergei Krutikov* Sergei Skvortsov* Alexander Samarin*

Abstract

Speculative decoding speeds up autoregressive generation in Large Language Models (LLMs) through a two-step procedure, where a lightweight draft model proposes tokens which the target model then verifies in a single forward pass. Although the drafter network is small in modern architectures, its LM-head still performs projection to a large vocabulary, becoming one of the major computational bottlenecks. In prior work this issue has been predominantly addressed via static or dynamic vocabulary truncation. Yet mitigating the bottleneck, these methods bring in extra complexity, such as special vocabulary curation, sophisticated inference-time logic or modifications of the training setup. In this paper, we propose **SlimSpec**, a low-rank parameterization of the drafter’s LM-head that compresses the inner representation rather than the output, preserving full vocabulary support. We evaluate our method with EAGLE-3 drafter across three target models and diverse benchmarks in both latency- and throughput-bound inference regimes. SlimSpec achieves $4\text{--}5\times$ acceleration over the standard LM-head architecture while maintaining a competitive acceptance length, surpassing existing methods by up to 8-9% of the end-to-end speedup. Our method requires minimal adjustments of training and inference pipelines. Combined with the aforementioned speedup improvements, it makes SlimSpec a strong alternative across wide variety of draft LM-head architectures.

1 Introduction

In the past years, Large Language Models (LLMs) have achieved strong performance across a wide range of tasks, but their autoregressive nature remains computationally inefficient at inference due to sequential token generation. As a result, latency and serving costs have become significant challenges for practical deployment. A central direction for mitigating these costs is *speculative decoding* [1, 2] that employs a lightweight draft model to propose multiple consecutive tokens which the target model then verifies in parallel. This procedure accelerates generation by sampling multiple tokens per speculative round on average, while preserving the output distribution of the target model.

Since its introduction, speculative decoding has evolved into a broad family of methods. Early approaches used standalone drafters, including pretrained small language models from the same model family or simple n-gram drafters that derive proposals from corpus statistics or the prompt context [3, 4]. More recent methods, such as MEDUSA, Hydra and the EAGLE family [5–9], integrate a lightweight drafter module into the target model directly, building it upon extracted hidden representations.

This design has become a commonly used approach due to lower overhead and higher acceptance quality, resulting into substantial improvements in the end-to-end speedups. One of its major bottlenecks, which limits further speedup advances, is computation of the draft token logits [10, 11].

*Nebius, Amsterdam, Netherlands.

Correspondence to: astr1rd@nebius.com

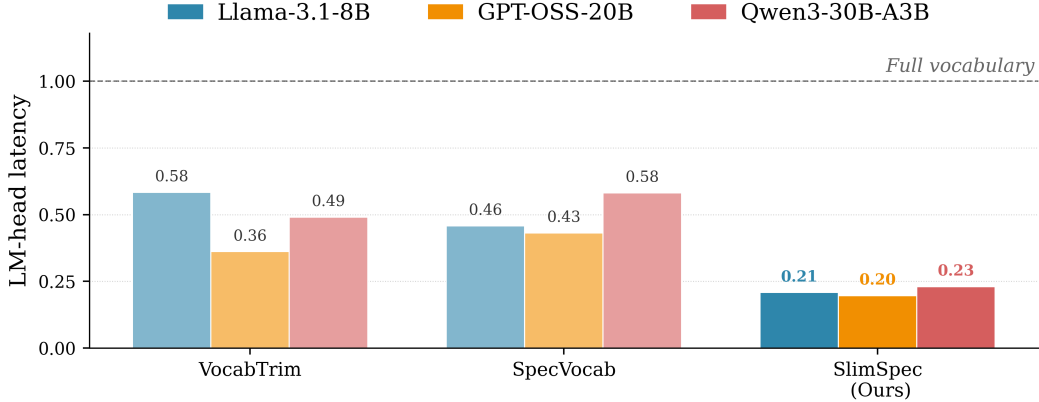


Figure 1: Relative LM-head GPU time T_{head} for batch size 1 across models; lower is better. The underlying T_{head} values are normalized with respect to the full vocabulary baseline, set to 1.0. VocabTrim reduces the draft vocabulary to 64K tokens. For SpecVocab and SlimSpec the low rank is set to $r = d/8$, where d is target model hidden size. Both VocabTrim and SpecVocab can reduce LM-head latency by only about 60%, while SlimSpec achieves an approximately 4-5 $\times$ reduction.

Although the drafter backbone can be small, its LM-head has to produce logits over the whole target model vocabulary whose size in modern LLMs often exceeds the order of 10^5 . This requires a large output projection at every drafted position, making the LM-head a natural computational bottleneck.

Existing methods mainly mitigate the aforementioned issue by shrinking the active vocabulary, either statically [10, 12, 13] or dynamically [11, 14, 15], thereby reducing the output projection along the vocabulary dimension. While being effective, these methods introduce additional complexity, such as vocabulary curation, token-index bookkeeping, inference-time routing or top- k selection.

In this paper, we explore a different direction of addressing the LM-head bottleneck. Instead of reducing the set of candidate tokens, we compress the hidden representation being used for logits prediction. Our approach preserves the full vocabulary of the target model, keeps computations dense and needs minimal changes in both training and inference pipelines. Our contributions are as follows:

- We propose **SlimSpec**, a drop-in low-rank LM-head architecture for speculative drafters. It replaces the standard LM-head with a factorized projection that compresses the hidden representation rather than the output vocabulary. We rigorously show that our approach is free of theoretical drawbacks inherent to vocabulary reduction methods, including hard ceiling on the acceptance rate and train-test mismatch of the target model distribution.
- We derive an acceptance-cost framework that reveals when LM-head acceleration translates into the end-to-end speedup. Our analysis establishes the relation between computational speedup and acceptance preservation, which helps finding a reasonable trade-off for better overall performance and provides the ground for comparing different LM-head designs.
- We validate this analysis in a production-like inference setup using EAGLE-3 drafter across three target models (Llama3.1-8B, GPT-OSS-20B, Qwen3-30B-A3B), diverse benchmarks, different decoding temperatures and serving regimes. Under identical training pipelines and serving configurations, SlimSpec maintains acceptance length close to the full-vocabulary baseline while reducing LM-head latency by approximately 4-5 times as shown in Figure 1, surpassing other evaluated methods by 8-9% in terms of the end-to-end speedup.

2 Related Work

Recent work has increasingly focused on reducing the complexity of the drafter LM-head in speculative decoding. We classify existing methods into two families.

The first family reduces the drafter-side cost through *static vocabulary truncation*. FR-Spec [10] and VocabTrim [12] share the same core idea: they truncate draft prediction to a smaller token set, thereby

Table 1: **Complexity and design parameters of LM-head acceleration methods.** *No Vocabulary Pruning* indicates whether the full vocabulary is preserved. *No Top- k Overhead* indicates whether the LM head needs top- k token selection at inference. *Hyperparameters* lists configuration choices required by each method, beyond standard drafter training settings. *LM Head Complexity* gives asymptotic FLOPs required to compute LM head forward pass per drafted token, where $V = |\mathcal{V}|$ is the full vocabulary size and d is the drafter hidden-state dimension.

Method	No Vocabulary Pruning	No Top- k Overhead	Hyperparameters	LM Head Complexity
Full Vocab	✓	✓	—	$\mathcal{O}(Vd)$
FR-Spec [10] VocabTrim [12] BCL [13]	✗	✓	V_{tr} : truncated vocab size	$\mathcal{O}(V_{\text{tr}}d)$
CORAL [14]	✗	✗	N : # groups k : # activated groups	$\mathcal{O}(d^2 + Nd + k\frac{Vd}{N})$
DynaSpec [15]	✗	✗	B : shortlist size r : router interm. size M : # clusters	$\mathcal{O}(2rd + rM + Bd)$
SpecVocab [11]	✓	✗	r : router low rank k : # selected tokens	$\mathcal{O}(rd + Vr + kd)$
SlimSpec (ours)	✓	✓	r : low rank	$\mathcal{O}(rd + Vr)$

reducing the cost of the drafter LM-head. The main difference lies in the source of the frequency statistics used to choose this truncated vocabulary. VocabTrim ranks tokens by their frequency in target-model sampled generations whereas FR-Spec ranks tokens by their occurrence frequency in a general-purpose text corpus. More recently, BCL [13] studied static truncation as an optimization problem that balances token coverage against draft-side latency through the choice of vocabulary size. Unlike VocabTrim and FR-Spec, BCL explicitly trains the drafter with the found optimal vocabulary, thereby aligning training and inference. Similarly, [16] also train draft models with truncated output vocabularies, although their main focus is the training objective rather than vocabulary selection.

The advantage of this family is architectural simplicity, but its limitation is inherent to vocabulary truncation. All tokens outside this vocabulary are assigned zero probability and can never be proposed by the drafter, which typically reduces acceptance quality. Recent work [17] mitigates this limitation by redistributing drafter probability mass toward target tokens outside the truncated vocabulary. However, this primarily serves as an acceptance-recovery mechanism for pruned vocabularies, rather than a method for making the LM-head computation itself cheaper.

The second family of methods perform *dynamic selection* of the active vocabulary subset. CORAL [14] and DynaSpec [15] both rely on a predefined partition of the vocabulary into small disjoint subsets and add a routing mechanism that selects a few active subsets for each context. The logits are then computed only over these selected subsets, reducing LM-head cost while allowing the active support to depend on the current context. SpecVocab [11] follows a related routing-based approach but avoids predefined expert sub-vocabularies. Instead of routing to these fixed partitions, it uses a learned low-rank router to predict a context-dependent token subset directly.

Compared with static vocabulary truncation, these methods can improve a trade-off between the speedup and acceptance quality by adapting the candidate vocabulary to the context. However, this flexibility comes at the cost of a more sophisticated design and implementation. As shown in Table 1, these methods introduce more hyperparameters and require an explicit top- k -style selection step before the final logit computation. The latter can become a noticeable bottleneck on GPUs because it involves such operations as global ranking, partial sorting, irregular indexing and gathering a context-dependent subset of weights, which are less efficient than dense matrix multiplication.

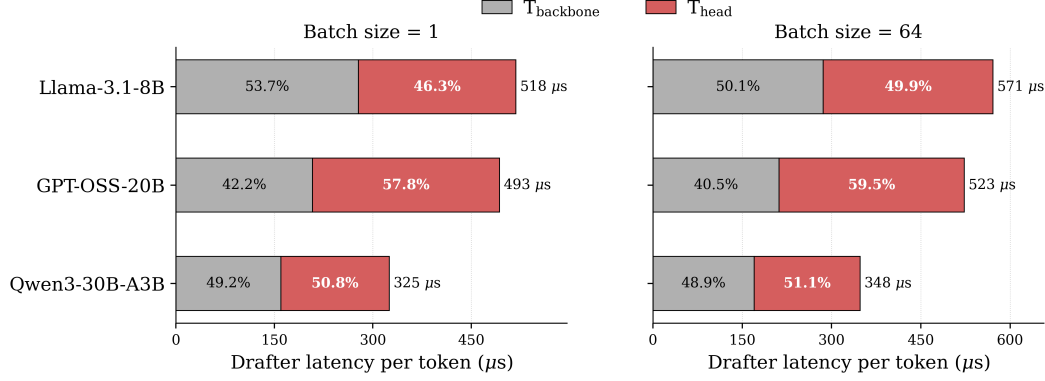


Figure 2: Drafter latency decomposition at batch sizes 1 and 64 for EAGLE-3 with full-vocabulary projection. Each horizontal bar breaks down T_{draft} into main block computation time T_{backbone} and LM-head computation time T_{head} for each target model, measured in microseconds.

We also note a broader line of work that compresses the LM-head via low-rank architectural factorization in standard language modeling [18–24]. These methods are not specific to speculative decoding and operate on a single model, so they are not applicable to the drafter-target setup considered here.

3 Performance Model for Draft LM-Head Acceleration

In this section, we analyze the throughput structure of speculative decoding and quantify the contribution of the drafter LM-head to the drafting cost. We also derive an acceptance–cost trade-off that governs how reducing this cost translates into end-to-end speedup.

3.1 Throughput structure of speculative decoding

Let n be the maximum number of draft tokens proposed per speculative step. Following the standard convention [1], we measure acceptance quality by the *average acceptance length* defined as

$$\tau = n \times \frac{\# \text{accepted tokens}}{\# \text{drafted tokens}} + 1. \quad (1)$$

Here, the first term estimates the average number of accepted draft tokens per speculative step, while the 1 accounts for the bonus token sampled from the target-model distribution after verification.

Let T_{draft} be the wall-clock time required to generate the draft tokens, T_{verify} be the wall-clock time of the target-model verification pass, and T_{overhead} be the pipeline overhead, including scheduling, synchronization, and cache management. Then the decoding throughput can be written [1, 14, 15] as the average number of *tokens per second*

$$\text{TPS} = \frac{\tau}{T_{\text{overhead}} + T_{\text{verify}} + T_{\text{draft}}}. \quad (2)$$

We further focus on modern auxiliary-head drafter architectures, such as MEDUSA, Hydra and the EAGLE family. Despite differences in their specific designs, their draft-side computation naturally separates into backbone components that produce draft hidden states and a final LM-head projection that maps these states to vocabulary logits. To isolate the LM-head cost T_{head} from the remaining computations T_{backbone} , we decompose

$$T_{\text{draft}} = T_{\text{backbone}} + T_{\text{head}}. \quad (3)$$

The main observation motivating our work is that T_{head} accounts for roughly **45%-60%** of T_{draft} , depending on the target model and inference regime (see Figure 2). The reason is structural: the draft model must remain lightweight in order to provide a speedup, yet at each drafted position it still has to produce a distribution over the entire target model vocabulary $\mathcal{V}$. The standard full-vocabulary projection has complexity $\mathcal{O}(Vd)$, where $V = |\mathcal{V}|$ is the vocabulary size and d is the hidden state dimension of the drafter. In modern LLMs, this corresponds to **hundreds of millions of operations per drafted token** and therefore makes the drafter LM-head a natural computational bottleneck.

3.2 Acceptance–cost trade-off

Reducing the LM-head cost is only useful if it translates into end-to-end speedup. For a method M^2 with mean acceptance length τ_M and head latency T_{head}^M , define

$$\rho_\tau = \frac{\tau_M}{\tau_{\text{Full}}}, \quad \nu = \frac{T_{\text{head}}^M}{T_{\text{head}}^{\text{Full}}}, \quad \kappa = \frac{T_{\text{head}}^{\text{Full}}}{T_{\text{non-head}}},$$

where $T_{\text{non-head}} = T_{\text{overhead}} + T_{\text{verify}} + T_{\text{backbone}}$. Here ρ_τ measures acceptance preservation, $\nu \in (0, 1]$ determines LM-head acceleration, and κ quantifies how much the LM-head dominates the rest of the speculation pipeline. Using the throughput formula (2), the end-to-end speedup of M relative to the full-vocabulary baseline is

$$\rho_{\text{TPS}}(\nu, \rho_\tau; \kappa) = \rho_\tau \cdot \frac{1 + \kappa}{1 + \nu\kappa}. \quad (4)$$

Equation (4) defines a family of speedup level curves on the (ν, ρ_τ) plane, parameterized by κ . A method with parameters (ν, ρ_τ) provides end-to-end speedup improvement over the full-vocabulary baseline if and only if

$$\rho_\tau > \frac{1 + \nu\kappa}{1 + \kappa}. \quad (5)$$

The right-hand side defines the minimum acceptance ratio which method M must preserve in order to convert its LM-head savings into end-to-end gains. If the LM-head accounts for a small fraction of pipeline costs, $\kappa \rightarrow 0$, the threshold approaches 1 and any acceptance loss is fatal. When the LM-head dominates, the threshold becomes smaller and more substantial acceptance loss is tolerated.

Parameter κ is not a property of the drafter alone but of the full deployment configuration. Larger or deeper target models likewise increase T_{verify} and lower κ . Batch size can switch individual pipeline components between memory- and compute-bound regimes, shifting the relative weight of $T_{\text{head}}^{\text{Full}}$ and $T_{\text{non-head}}$. Sampling temperature also plays a role: stochastic decoding requires a softmax over the whole vocabulary, increasing T_{overhead} and thereby reducing κ relative to greedy decoding. Other factors include sampling tokens from the residual distribution, computing acceptance probabilities and performing stochastic rejection sampling itself. Finally, since the standard LM-head scales as $\mathcal{O}(Vd)$ while the rest of the drafter scales with d^2 , target models with larger vocabularies (relative to d) push κ upward.

4 SlimSpec

The framework introduced in Section 3.2 establishes the condition when LM-head acceleration translates to end-to-end speedup improvements. LM head must achieve a sufficiently low latency factor ν without sacrificing too much acceptance ratio ρ_τ , with the exact trade-off governed by the parameter κ . In this section, we introduce SlimSpec, a lightweight LM-head architecture for speculative decoding, whose design is driven by the aforementioned principles.

4.1 LM-head architecture

Let $\mathbf{h} \in \mathbb{R}^d$ denote the hidden representation produced by the draft model backbone, and $\mathbf{z} \in \mathbb{R}^V$ be the corresponding logits. The standard full-vocabulary projection is

$$\mathbf{z} = \mathbf{W}_{\text{full}} \mathbf{h}, \quad \mathbf{W}_{\text{full}} \in \mathbb{R}^{V \times d}.$$

SlimSpec replaces it with the low-rank factorization

$$\mathbf{z} = \mathbf{W}_{\text{up}} \mathbf{W}_{\text{down}} \mathbf{h}, \quad \mathbf{W}_{\text{down}} \in \mathbb{R}^{r \times d}, \quad \mathbf{W}_{\text{up}} \in \mathbb{R}^{V \times r},$$

²By the method M we mean a particular design of the draft LM head, with $M = \text{Full}$ being a standard LM-head projection to the full target vocabulary. Accordingly, τ_M and T_{head}^M correspond to the respective quantities measured using identical setups except different LM heads.

where $r < d$ is the chosen rank³. The full target vocabulary is preserved through W_{up} , while LM-head computational cost reduces from $\mathcal{O}(Vd)$ to $\mathcal{O}(rd + Vr)$. Since $V \gg d$ in modern LLMs, the cost reduction (in FLOPs) is approximately linear in r :

$$\frac{rd + Vr}{Vd} = \frac{r}{d} \cdot \left(1 + \frac{d}{V}\right) \approx \frac{r}{d}.$$

Conceptually, the vocabulary is not trimmed - instead, the hidden representation used for logits prediction is compressed. This is the main distinction between SlimSpec and vocabulary-truncation approaches: it keeps all V token logits available while generating them via a thinner representation.

The rank r is the only architectural hyperparameter of SlimSpec, which positively distinguishes it from dynamic vocabulary truncation methods. It controls both the width of the compressed hidden state and the computational cost of the head. In practice, useful ranks are fractions of the drafter hidden dimension, such as $d/4$, $d/8$ or $d/16$. We further study this trade-off empirically in Section 6.

4.2 Advantages over vocabulary truncation

The central design decision in SlimSpec is compressing the hidden representation rather than restricting the output support. We argue here that this choice is structurally superior to vocabulary truncation due to two key properties: output support preservation and a train-test consistency.

Acceptance upper bound Let p and q be the target and draft distributions respectively at a given draft position. The acceptance rate for this position is governed by the distributional overlap

$$\alpha = \sum_{v \in \mathcal{V}} \min(p(v), q(v)).$$

If the drafter is restricted to a truncated vocabulary $\mathcal{V}_{\text{tr}} \subset \mathcal{V}$, then $q(v) = 0$ for $v \notin \mathcal{V}_{\text{tr}}$, which implies

$$\alpha \leq \sum_{v \in \mathcal{V}_{\text{tr}}} p(v)$$

for *any* draft distribution q ⁴. This bound holds for all drafters with the truncated vocabulary, regardless of training quality, parameter count, routing scheme or alike. SlimSpec is not subject to this bound.

Train-test mismatch under KL training A more subtle problem arises when vocabulary truncation is combined with a forward Kullback-Leibler divergence loss during training of the drafter. Let $\mathbf{z}_p \in \mathbb{R}^V$ be the logits of the target model, so $p = \text{softmax}(\mathbf{z}_p)$. When the drafter’s LM-head is restricted to $\mathcal{V}_{\text{tr}}$, the KL divergence becomes infinite since $q(v) = 0, \forall v \notin \mathcal{V}_{\text{tr}}$, whilst $p(v) > 0$ for all finite logits. In practice, this inconsistency is resolved by redefining the target as

$$\tilde{p}(v) = \text{softmax}(\mathbf{m} \odot \mathbf{z}_p),$$

where mask $\mathbf{m}$ sets logits $\mathbf{z}_p$ to $-\infty$ for the tokens outside $\mathcal{V}_{\text{tr}}$ [16].

This introduces a discrepancy between target distributions being used in the training objective and in the test-time acceptance logic. At inference, the drafter is verified against the full target distribution p whereas during training it only sees a truncated and re-normalized distribution $\tilde{p}$. Therefore, KL-based training is likely to produce overconfident draft probabilities on $\mathcal{V}_{\text{tr}}$ at large scale, which may harm acceptance rates since overshooting test-time target probabilities reduces acceptance probabilities.

Simplicity Unlike vocabulary truncated methods, SlimSpec does not require complex data preprocessing to compute token statistics or storing and manipulating token index mappings. It only consists of regular dense matrix multiplications, avoiding poorly scalable routing or top- k -selection logic. SlimSpec requires small modification of the LM head and can be seamlessly plugged into any existing drafter without altering its backbone or training pipeline. This makes our method substantially easier to implement than competing approaches.

³Although our parameterization might resemble a well-known LoRA-style adapters, its role is different. LoRA adds a low-rank update to an existing full-rank matrix and therefore preserves the full-rank path. In contrast, SlimSpec completely removes the full-rank LM-head and replaces it by the low-rank representation.

⁴Under greedy decoding ($T = 0$), target distribution becomes a point-mass on $v^* = \arg \max_{v \in \mathcal{V}} p(v)$, so the bound reduces to $\alpha \leq \mathbf{1}[v^* \in \mathcal{V}_{\text{tr}}]$ and acceptance collapses to 0 whenever $v^* \notin \mathcal{V}_{\text{tr}}$.

5 Experimental Settings

5.1 Methods

We compare SlimSpec, as it is introduced in Section 4, against three groups of baselines described below. As the default, we use a standard approach that performs a linear projection to the full target vocabulary. We further refer to this baseline as Full Vocab.

For static vocabulary truncation, we consider two post-training baselines, VocabTrim and FR-Spec. Following the methodology of the original papers, we select a token subset by ranking vocabulary according to the frequency statistics collected on the calibration dataset. In VocabTrim it is simply the training dataset whereas in FR-Spec it is a general-purpose SlimPajama-627B [25].

As a training-aware baseline, we consider BCL which performs vocabulary truncation according to the optimal coverage-latency trade-off. We also report VocabTrim-T which trains the drafter using the same truncated vocabulary as VocabTrim.

For dynamic vocabulary truncation, we consider SpecVocab due to its simplicity and strong performance. We implement this method following the original paper and report results for several values of the router rank r .

5.2 Training configuration

We conduct experiments across three target models: Llama-3.1-8B-Instruct [26], GPT-OSS-20B [27], and Qwen3-30B-A3B-Instruct-2507 [28]. We construct the training corpus from 660K prompts from Infinity-Instruct-0625 dataset [29] by generating responses with the corresponding target model. This ensures that the drafter training data distribution matches the target-model samples encountered at inference time.

We choose EAGLE-3 [9] setup as the best-performing state-of-the-art draft training pipeline. All drafters are trained with $n = 6$ speculative tokens with the weights shared across positions. Draft backbone architecture is fixed for each target model, so the methods in the study differ only in the LM-head design. We employ standard KL-divergence loss as our training objective, unless stated otherwise. More details on architectures, training hyperparameters and loss specifications are provided in Appendix A.

5.3 Evaluation protocol

We evaluate all methods across three benchmarks: **MT-Bench** [30], **HumanEval** [31], and **GSM8K** [32], covering such domains as instruction following, code generation and mathematical reasoning respectively. The evaluations are performed under both greedy (temperature = 0) and stochastic decoding (temperature = 1) with batch sizes 1 and 64, corresponding to latency-critical and high-throughput serving regimes. We use production-like inference environment based on vLLM 0.17.1 with NVIDIA H200 GPUs.

As our primary metric, we select generation throughput measured in tokens per second (TPS), which captures the end-to-end serving efficiency of each speculative decoding variant. To assess drafter quality independently of raw throughput, we additionally report average acceptance length τ as defined by (1). Each reported speedup value and τ is obtained by averaging over 5 identical runs with different random seeds.

6 Evaluation Results

As discussed in Section 3.2, the efficiency of a draft-head design is governed by the trade-off between acceptance preservation ρ_τ and relative LM-head cost ν . In this section, we compare methods outlined in 5.1 with Llama-3.1-8B as the target model and analyze their acceptance-cost trade-off in the (ν, ρ_τ) plane. Figure 3 plots the results for a representative subset of methods, with corresponding numerical values for temperature 0 at batch size 1 being reported in Appendix C.

Static vocabulary truncation (VocabTrim, FR-Spec, BCL) faces a clear cost-acceptance frontier: smaller vocabularies reduce ν but degrade ρ_τ proportionally. FR-Spec is surpassed by VocabTrim

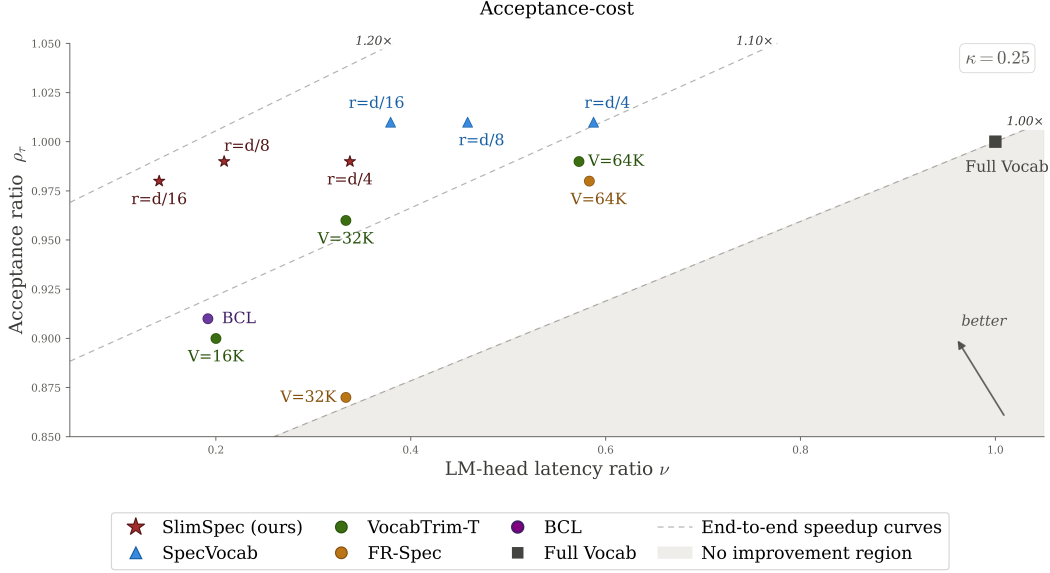


Figure 3: End-to-end speedup decomposition in the (ν, ρ_τ) plane for Llama-3.1-8B with temperature 0 at batch size 1 ($\kappa = 0.25$). Dashed lines are theoretical speedup level curves derived from equation 4. The shaded region indicates no end-to-end improvement over the full-vocabulary baseline. SlimSpec (red stars) achieves the largest LM-head acceleration while keeping ρ_τ close to 1.

because its frequency statistics, computed on a generic corpus, are less aligned with target-model generations than statistics collected on actual training data. BCL selects too aggressive truncation strategy whose acceptance preservation is not compensated by the LM-head cost reduction. Training-aware truncation (VocabTrim-T) demonstrates similar results to post-training truncation (VocabTrim) at any of the evaluated vocabulary sizes. For clarity we plot only VocabTrim-T in Figure 3.

Dynamic truncation with SpecVocab pushes the frontier upward, outperforming the static truncation baselines as expected. With ranks $r = d/8$ and $r = d/16$ it preserves acceptance of the Full Vocab ($\rho_\tau \approx 1$) while reducing LM-head latency approximately by 60%.

SlimSpec surpasses both these families. With rank $r = d/8$ it reaches approximately $5\times$ reduction in LM-head cost maintaining sufficiently high acceptance quality $\rho_\tau = 0.99$. We adopt $r = d/8$ as the default SlimSpec configuration in the rest of the evaluation.

Finally, we compare SlimSpec against the strongest representative of each baseline family: Full Vocab, VocabTrim-T with $V_{tr} = 64K$, and SpecVocab with $r = d/8$. Comparison summary is presented in Table 2, with detailed results being reported in Appendix D. Averaged across benchmarks, it improves by more than 8.5% over the static truncation baseline for Llama-3.1-8B with both batch sizes 1 and 64, and by 8.9% over SpecVocab on GPT-OSS-20B at batch size 64. For Qwen3-30B-A3B T_{head} accounts for a smaller fraction of the total latency, decreasing κ significantly and shrinking the observed speedup to at most 1-2% over the strongest baseline. Overall, SlimSpec is never worse than the baselines in most setups, requiring no vocabulary curation or additional inference-time logic.

7 Conclusion

In this work, we study different approaches to draft LM-head acceleration through the lens of the acceptance-cost trade-off rather than through the head latency alone. Our performance model shows that a computationally cheaper head does not necessarily improve the end-to-end throughput. This happens when the reduction in head cost is not offset by a loss in the average number of accepted draft tokens. This perspective explains why vocabulary reduction methods require a careful design choice. They make the final projection cheaper, but risk deteriorating acceptance quality or introducing new overheads, which limits the realized throughput gain.

Table 2: Comparison of SlimSpec with the main baselines for three target models on three benchmarks at temperature 0. Speedup is measured relative to vanilla inference without a speculator. MT, HE, and GSM denote MT-Bench, HumanEval, and GSM8K, respectively. Avg denotes the arithmetic mean across these three benchmarks for each batch size.

Method	Batch size = 1				Batch size = 64			
	MT	HE	GSM	Avg	MT	HE	GSM	Avg
Llama-3.1-8B-Instruct								
Full Vocab	2.23	2.78	2.42	2.48	1.35	1.43	1.41	1.40
VocabTrim-T	2.47	2.96	2.66	2.70	1.29	1.48	1.46	1.41
SpecVocab	2.59	3.17	2.82	2.86	1.36	1.50	1.52	1.46
SlimSpec	2.60	3.30	2.91	2.94	1.43	1.58	1.56	1.52
GPT-OSS-20B								
Full Vocab	1.43	1.41	1.63	1.49	1.36	1.17	1.38	1.30
VocabTrim-T	1.72	1.67	1.95	1.78	1.50	1.27	1.52	1.43
SpecVocab	1.74	1.71	1.98	1.81	1.43	1.18	1.45	1.35
SlimSpec	1.76	1.76	2.11	1.88	1.52	1.34	1.56	1.47
Qwen3-30B-A3B-Instruct								
Full Vocab	1.55	2.07	2.12	1.91	1.22	1.32	1.48	1.34
VocabTrim-T	1.66	2.15	2.23	2.01	1.31	1.36	1.53	1.40
SpecVocab	1.66	2.19	2.24	2.03	1.28	1.34	1.49	1.37
SlimSpec	1.66	2.22	2.27	2.05	1.31	1.36	1.54	1.40

We further present SlimSpec, an approach that follows a complementary direction, compressing the hidden representation predicted by the drafter backbone. It preserves full-vocabulary support and moves primarily along the cost axis, making the reduction in head latency more likely to translate into end-to-end gains. Our experiments confirm this intuition. In the acceptance-cost plane, SlimSpec occupies the most favorable region among the evaluated draft-head designs, combining a substantially lower LM-head cost with acceptance close to the full-vocabulary baseline. In our test settings, the rank- $d/8$ configuration gives the best overall trade-off, achieving approximately $4\text{-}5\times$ LM-head acceleration and the strongest throughput among the compared methods.

In our vision, future work would focus less on further reducing the head cost and more on improving acceptance at the same cost. Therefore, acceptance-oriented training objectives are a natural candidate, since they could shift SlimSpec upward in the acceptance-cost plane without changing inference-time complexity. Another direction could be position-wise adaptivity, e.g. by sharing the vocabulary-side projection while using position-specific compression projections, or by assigning larger ranks to positions where additional capacity has the highest impact. Such extensions preserve the main advantage of SlimSpec while being driven by the acceptance-cost trade-off.

8 Limitations

The method proposed in this paper has a number of limitations, including: (i) the rank r is a manually chosen hyperparameter and we do not provide an automatic selection procedure; (ii) all experiments are conducted with EAGLE-3 draft heads, so the conclusions drawn in the paper transfer to other drafter families (e.g. MEDUSA, Hydra) only by analogy; (iii) measured speedups depend on the inference framework (vLLM 0.17.1) and hardware (NVIDIA H200) and may differ on other stacks; and (iv) some of the dynamic-vocabulary methods, namely CORAL and DynaSpec, are not considered due to the lack of reference implementations, which narrows the coverage of our study. We further discuss these limitations in detail in Appendix B.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2023. doi: 10.48550/arXiv.2211.17192. URL <https://arxiv.org/abs/2211.17192>.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. doi: 10.48550/arXiv.2302.01318. URL <https://arxiv.org/abs/2302.01318>.
- [3] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1582–1595, 2024.
- [4] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. In *Proceedings of the 41st International Conference on Machine Learning*, pages 14060–14079, 2024.
- [5] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, J. D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. doi: 10.48550/arXiv.2401.10774. URL <https://arxiv.org/abs/2401.10774>.
- [6] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusingha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024. doi: 10.48550/arXiv.2402.05109. URL <https://arxiv.org/abs/2402.05109>.
- [7] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. doi: 10.48550/arXiv.2401.15077. URL <https://arxiv.org/abs/2401.15077>.
- [8] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024. doi: 10.48550/arXiv.2406.16858. URL <https://arxiv.org/abs/2406.16858>.
- [9] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. doi: 10.48550/arXiv.2503.01840. URL <https://arxiv.org/abs/2503.01840>.
- [10] Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Ao Sun, Yuxiang Huang, Kaihuo Zhang, Weilin Zhao, Yuxuan Li, Jianyong Wang, Zhiyuan Liu, and Maosong Sun. Fr-spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. *arXiv preprint arXiv:2502.14856*, 2025. doi: 10.48550/arXiv.2502.14856. URL <https://arxiv.org/abs/2502.14856>.
- [11] Miles Williams, Young D. Kwon, Rui Li, Alexandros Kouris, and Stylianos I. Venieris. Speculative decoding with a speculative vocabulary. *arXiv preprint arXiv:2602.13836*, 2026. doi: 10.48550/arXiv.2602.13836. URL <https://arxiv.org/abs/2602.13836>.
- [12] Raghav Goel, Sudhanshu Agrawal, Mukul Gagrani, Junyoung Park, Yifan Zao, He Zhang, Tian Liu, Yiping Yang, Xin Yuan, Jiuyan Lu, Chris Lott, and Mingu Lee. Vocabtrim: Vocabulary pruning for efficient speculative decoding in llms. *arXiv preprint arXiv:2506.22694*, 2025. doi: 10.48550/arXiv.2506.22694. URL <https://arxiv.org/abs/2506.22694>.
- [13] Ofir Ben Shoham. Balancing coverage and draft latency in vocabulary trimming for faster speculative decoding. *arXiv preprint arXiv:2603.05210*, 2026. doi: 10.48550/arXiv.2603.05210. URL <https://arxiv.org/abs/2603.05210>.
- [14] Yepeng Weng, Dianwen Mei, Huishi Qiu, Xujie Chen, Li Liu, Jiang Tian, and Zhongchao Shi. Coral: Learning consistent representations across multi-step training with lighter speculative drafter. *arXiv preprint arXiv:2502.16880*, 2025. doi: 10.48550/arXiv.2502.16880. URL <https://arxiv.org/abs/2502.16880>.

- [15] Jinbin Zhang, Nasib Ullah, Erik Schultheis, and Rohit Babbar. Dynaspec: Context-aware dynamic speculative sampling for large-vocabulary language models. *arXiv preprint arXiv:2510.13847*, 2025. doi: 10.48550/arXiv.2510.13847. URL <https://arxiv.org/abs/2510.13847>.
- [16] Alexander Samarin, Sergei Krutikov, Anton Shevtsov, Sergei Skvortsov, Filipp Fisin, and Alexander Golubev. Lk losses: Direct acceptance rate optimization for speculative decoding. *arXiv preprint arXiv:2602.23881*, 2026. doi: 10.48550/arXiv.2602.23881. URL <https://arxiv.org/abs/2602.23881>.
- [17] Nadav Timor, Jonathan Mamou, Oren Pereg, Hongyang Zhang, and David Harel. Out-of-vocabulary sampling boosts speculative decoding. *arXiv preprint arXiv:2506.03206*, 2025. doi: 10.48550/arXiv.2506.03206. URL <https://arxiv.org/abs/2506.03206>.
- [18] Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou. Efficient softmax approximation for GPUs. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1302–1310, 2017.
- [19] Patrick H. Chen, Si Si, Yang Li, Ciprian Chelba, and Cho-Jui Hsieh. GroupReduce: Block-wise low-rank approximation for neural language model shrinking. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- [20] Alexei Baevski and Michael Auli. Adaptive input representations for neural language modeling. In *International Conference on Learning Representations (ICLR)*, 2019.
- [21] Vasileios Lioutas, Ahmad Rashid, Krtin Kumar, Md Akmal Haidar, and Mehdi Rezagholizadeh. Improving word embedding factorization for compression using distilled nonlinear neural decomposition. *arXiv preprint arXiv:1910.06720*, 2019. doi: 10.48550/arXiv.1910.06720. URL <https://arxiv.org/abs/1910.06720>.
- [22] Oleksii Hrinchuk, Valentin Khrulkov, Leyla Mirvakhabova, Elena Orlova, and Ivan Oseledets. Tensorized embedding layers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4847–4860, 2020.
- [23] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020.
- [24] Alaa Maalouf, Harry Lang, Daniela Rus, and Dan Feldman. Deep learning meets projective clustering. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=EQfpYwF3-b>.
- [25] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob Robert Steeves, Joel Hestness, and Nolan Dey. Slimpajama: A 627b token cleaned and deduplicated version of redpajama. *Cerebras Systems*, 2023. URL <https://www.cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>.
- [26] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. doi: 10.48550/arXiv.2407.21783. URL <https://arxiv.org/abs/2407.21783>.
- [27] Sandhini Agarwal et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. doi: 10.48550/arXiv.2508.10925. URL <https://arxiv.org/abs/2508.10925>.
- [28] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. doi: 10.48550/arXiv.2505.09388. URL <https://arxiv.org/abs/2505.09388>.
- [29] Jijie Li, Li Du, Hanyu Zhao, Bo-wen Zhang, Liangdong Wang, Boyan Gao, Guang Liu, and Yonghua Lin. Infinity instruct: Scaling instruction selection and synthesis to enhance language models. *arXiv preprint arXiv:2506.11116*, 2025. doi: 10.48550/arXiv.2506.11116. URL <https://arxiv.org/abs/2506.11116>.

- [30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023. doi: 10.48550/arXiv.2306.05685. URL <https://arxiv.org/abs/2306.05685>.
- [31] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- [32] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168. URL <https://arxiv.org/abs/2110.14168>.

A Training Configurations

All draft models are trained for 10 epochs with batch size 64 and learning rate 4×10^{-4} . We use AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, and no weight decay. The learning rate is scheduled with a cosine decay after 100 warmup steps, and gradients are clipped to a maximum norm of 0.5.

Each drafter follows the EAGLE-3-style architecture with $n = 6$ decoding heads; the models differ only in the design of the LM-head. All drafters except SpecVocab are trained with the forward KL objective:

$$\text{KL}(p||q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{p(v)}{q(v)},$$

where p and q are target and draft model distributions respectively. For VocabTrim-T and BCL we re-normalize p and take the sum over $\mathcal{V}_{\text{tr}}$. For SpecVocab, we follow the objective from the corresponding paper and add an auxiliary KL term with weight coefficient $\lambda = 0.01$.

EAGLE-3 draft backbone consists of a single dense transformer layer that mirrors the target model’s architecture. For MoE target models the intermediate dimension of the feed-forward network is chosen as

$$d_{\text{ffn}} = \text{num-experts-per-block} \times d_{\text{expert}},$$

where num-experts-per-block is the number of experts activated per token and d_{expert} is the intermediate dimension of each expert’s FFN.

Training a single drafter requires 128 GPU-hours for Llama-3.1-8B-Instruct, 288 GPU-hours for GPT-OSS-20B, and 320 GPU-hours for Qwen3-30B-A3B-Instruct. All computations are performed on NVIDIA H200 GPUs.

B Limitations discussion

Our analysis and experiments focus on the regime where the draft LM head is a non-negligible part of the speculative decoding pipeline. The acceptance-cost model in Section 3 shows that LM-head acceleration translates into end-to-end speedup only when the saved head latency is large enough relative to the rest of the pipeline. Consequently, the gains of SlimSpec are dependent on the deployment setup. Serving stacks, GPU kernels, batch size, draft verification costs, sampling overhead and scheduler behavior can all change the effective value of κ and therefore the realized throughput improvement.

Our empirical study is also limited to EAGLE-3 auxiliary drafters. SlimSpec is a local replacement of the draft LM head and should be compatible with other auxiliary-drafter families such as MEDUSA or Hydra, but we do not evaluate those architectures directly. Similarly, all main experiments use a fixed draft-backbone architecture and shared weights across draft positions. We therefore do not characterize how the low-rank head interacts with alternative drafter backbones, dynamic draft trees, position-specific heads or standalone draft models.

The evaluation covers three target models and three benchmarks, but it does not exhaust the range of possible deployment scenarios. We do not study really large target models (e.g. $> 50\text{B}$ parameters), long-context generation, multilingual or domain-specific workloads, tool-use settings, high sampling temperatures, alternative sampling policies, or other hardware and serving frameworks. Since the acceptance-cost trade-off depends on both the target distribution and the inference stack, the numerical speedups reported here should be interpreted as measurements for the evaluated production-like setup rather than hardware-independent constants.

The rank r is chosen manually, and we evaluate several simple fractions of the hidden dimension, finding that $r = d/8$ gives the best trade-off in our tested settings. However, selecting the best rank depends on many factors, including the target model, vocabulary size, serving regime and benchmark distribution. We do not provide an automatic rank-selection procedure or recommendations how to determine the optimal rank besides thorough hyperparameter tuning.

Finally, our comparison focuses on representative methods that we could implement faithfully under a shared training and inference pipeline. We include static truncation methods and SpecVocab as a strong dynamic-vocabulary baseline, but we do not reproduce CORAL or DynaSpec because they require additional machinery, while no reference implementation was available to us. More optimized kernels for dynamic vocabulary selection could change the relative overheads of these methods.

C Detailed acceptance–cost results

Table 3: Acceptance–cost trade-off for all considered draft-head designs with Llama-3.1-8B at temperature 0 and batch size 1. The table reports values averaged over three datasets: MT-Bench, HumanEval, and GSM8K. Speedup is measured relative to the full-vocabulary baseline; ρ_τ denotes acceptance preservation, and ν denotes relative LM-head cost.

Method	Configurations	Batch size = 1		
		Speedup	ρ_τ	ν
Full Vocab	–	1.00	1.00	1.00
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.06	0.98	0.58
	$V_{\text{tr}} = 32\text{K}$	1.01	0.87	0.33
	$V_{\text{tr}} = 16\text{K}$	0.93	0.78	0.20
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.08	0.99	0.58
	$V_{\text{tr}} = 32\text{K}$	1.10	0.96	0.33
	$V_{\text{tr}} = 16\text{K}$	1.08	0.90	0.20
BCL	$V_{\text{tr}} = 15.8\text{K}$	1.07	0.91	0.19
	$V_{\text{tr}} = 64\text{K}$	1.09	1.00	0.58
VocabTrim-T	$V_{\text{tr}} = 32\text{K}$	1.09	0.96	0.33
	$V_{\text{tr}} = 16\text{K}$	1.07	0.90	0.20
SpecVocab	$r = d/4$	1.13	1.01	0.59
	$r = d/8$	1.16	1.01	0.46
	$r = d/16$	1.15	1.01	0.38
SlimSpec	$r = d/4$	1.16	0.99	0.34
	$r = d/8$	1.19	0.99	0.21
	$r = d/16$	1.18	0.98	0.14

D Extended Results Tables

We report vLLM speedup and average acceptance length measurements for all considered LM-head acceleration methods and their different configurations. The *Config* column reports the corresponding hyperparameter for each method: it corresponds to the vocabulary size V_{tr} for methods with a static truncated vocabulary; , the intermediate router rank r for SpecVocab; and the intermediate LM-head rank for SlimSpec. *Speedup* is measured relative to vanilla inference without a speculator, under the same benchmark, temperature, and batch size.

We provide 12 tables in total, covering all combinations of models, temperature settings, and batch sizes. For each benchmark and each configuration, we run the evaluation 5 times. We report Mean $\pm$ standard deviation over 5 these runs. **Bold** marks the best value among compressed methods, excluding *Full Vocab*, which serves as the upper-bound reference. *Avg* denotes the arithmetic mean across these three benchmarks for each batch size.

Table 4: Speedup for Llama3.1-8B-Instruct, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	2.23 ± 0.02	2.78 ± 0.01	2.42 ± 0.01	2.48
FR-Spec	$V_{tr} = 64K$	2.35 ± 0.01	2.93 ± 0.02	2.59 ± 0.01	2.62
	$V_{tr} = 32K$	2.27 ± 0.03	2.74 ± 0.02	2.47 ± 0.01	2.50
	$V_{tr} = 16K$	2.09 ± 0.01	2.47 ± 0.01	2.34 ± 0.01	2.30
VocabTrim	$V_{tr} = 64K$	2.42 ± 0.02	3.01 ± 0.02	2.63 ± 0.02	2.69
	$V_{tr} = 32K$	2.50 ± 0.02	2.98 ± 0.02	2.66 ± 0.03	2.71
	$V_{tr} = 16K$	2.44 ± 0.01	2.91 ± 0.02	2.63 ± 0.02	2.66
BCL	$V_{tr} = 15.8K$	2.45 ± 0.01	2.89 ± 0.02	2.63 ± 0.00	2.65
VocabTrim-T	$V_{tr} = 64K$	2.47 ± 0.03	2.96 ± 0.02	2.66 ± 0.01	2.70
	$V_{tr} = 32K$	2.50 ± 0.02	2.96 ± 0.02	2.65 ± 0.01	2.70
	$V_{tr} = 16K$	2.43 ± 0.05	2.88 ± 0.03	2.64 ± 0.02	2.65
SpecVocab	$r = d/4$	2.50 ± 0.03	3.13 ± 0.02	2.74 ± 0.01	2.79
	$r = d/8$	2.59 ± 0.02	3.17 ± 0.02	2.82 ± 0.01	2.86
	$r = d/16$	2.59 ± 0.03	3.16 ± 0.03	2.80 ± 0.03	2.85
SlimSpec	$r = d/4$	2.58 ± 0.02	3.22 ± 0.02	2.81 ± 0.02	2.87
	$r = d/8$	2.60 ± 0.03	3.30 ± 0.03	2.91 ± 0.02	2.94
	$r = d/16$	2.61 ± 0.01	3.29 ± 0.03	2.90 ± 0.00	2.93

Table 5: Average acceptance length τ for Llama3.1-8B-Instruct, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.93 ± 0.00	4.93 ± 0.02	4.40 ± 0.00	4.42
FR-Spec	$V_{tr} = 64K$	3.81 ± 0.00	4.80 ± 0.02	4.33 ± 0.00	4.31
	$V_{tr} = 32K$	3.46 ± 0.01	4.23 ± 0.01	3.89 ± 0.00	3.86
	$V_{tr} = 16K$	3.08 ± 0.01	3.71 ± 0.00	3.59 ± 0.00	3.46
VocabTrim	$V_{tr} = 64K$	3.92 ± 0.00	4.89 ± 0.03	4.38 ± 0.00	4.40
	$V_{tr} = 32K$	3.84 ± 0.01	4.64 ± 0.02	4.26 ± 0.00	4.25
	$V_{tr} = 16K$	3.60 ± 0.01	4.33 ± 0.02	4.04 ± 0.00	3.99
BCL	$V_{tr} = 15.8K$	3.64 ± 0.01	4.34 ± 0.01	4.08 ± 0.01	4.02
VocabTrim-T	$V_{tr} = 64K$	3.95 ± 0.01	4.88 ± 0.01	4.37 ± 0.01	4.40
	$V_{tr} = 32K$	3.86 ± 0.01	4.61 ± 0.01	4.24 ± 0.01	4.24
	$V_{tr} = 16K$	3.63 ± 0.02	4.29 ± 0.01	4.04 ± 0.00	3.99
SpecVocab	$r = d/4$	3.94 ± 0.01	4.95 ± 0.01	4.45 ± 0.00	4.45
	$r = d/8$	3.96 ± 0.02	4.93 ± 0.02	4.47 ± 0.01	4.45
	$r = d/16$	3.99 ± 0.02	4.93 ± 0.03	4.45 ± 0.00	4.46
SlimSpec	$r = d/4$	3.90 ± 0.03	4.87 ± 0.03	4.39 ± 0.00	4.39
	$r = d/8$	3.84 ± 0.02	4.89 ± 0.01	4.39 ± 0.00	4.37
	$r = d/16$	3.79 ± 0.01	4.83 ± 0.03	4.37 ± 0.01	4.33

Table 6: Speedup for Llama3.1-8B-Instruct, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.35 ± 0.01	1.43 ± 0.01	1.41 ± 0.02	1.39
FR-Spec	$V_{tr} = 64K$	1.35 ± 0.02	1.49 ± 0.03	1.45 ± 0.02	1.43
	$V_{tr} = 32K$	1.33 ± 0.01	1.45 ± 0.01	1.41 ± 0.02	1.40
	$V_{tr} = 16K$	1.19 ± 0.01	1.32 ± 0.02	1.35 ± 0.02	1.29
VocabTrim	$V_{tr} = 64K$	1.37 ± 0.02	1.51 ± 0.02	1.49 ± 0.03	1.46
	$V_{tr} = 32K$	1.39 ± 0.04	1.44 ± 0.03	1.49 ± 0.02	1.44
	$V_{tr} = 16K$	1.35 ± 0.02	1.46 ± 0.04	1.48 ± 0.02	1.43
BCL	$V_{tr} = 15.8K$	1.37 ± 0.02	1.48 ± 0.02	1.47 ± 0.01	1.44
VocabTrim-T	$V_{tr} = 64K$	1.29 ± 0.03	1.48 ± 0.03	1.46 ± 0.01	1.41
	$V_{tr} = 32K$	1.29 ± 0.03	1.48 ± 0.03	1.45 ± 0.02	1.41
	$V_{tr} = 16K$	1.37 ± 0.03	1.49 ± 0.02	1.46 ± 0.03	1.44
SpecVocab	$r = d/4$	1.36 ± 0.02	1.39 ± 0.20	1.44 ± 0.05	1.40
	$r = d/8$	1.36 ± 0.04	1.50 ± 0.04	1.52 ± 0.04	1.46
	$r = d/16$	1.36 ± 0.03	1.50 ± 0.03	1.48 ± 0.09	1.45
SlimSpec	$r = d/4$	1.41 ± 0.05	1.56 ± 0.04	1.55 ± 0.04	1.51
	$r = d/8$	1.43 ± 0.04	1.58 ± 0.05	1.56 ± 0.03	1.53
	$r = d/16$	1.41 ± 0.01	1.56 ± 0.05	1.50 ± 0.03	1.49

Table 7: Average acceptance length τ for Llama3.1-8B-Instruct, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.91 ± 0.01	4.90 ± 0.02	4.38 ± 0.03	4.40
FR-Spec	$V_{tr} = 64K$	3.76 ± 0.02	4.77 ± 0.02	4.31 ± 0.00	4.28
	$V_{tr} = 32K$	3.43 ± 0.02	4.22 ± 0.01	3.85 ± 0.02	3.83
	$V_{tr} = 16K$	3.06 ± 0.02	3.71 ± 0.01	3.57 ± 0.02	3.45
VocabTrim	$V_{tr} = 64K$	3.88 ± 0.04	4.89 ± 0.02	4.35 ± 0.02	4.37
	$V_{tr} = 32K$	3.80 ± 0.02	4.63 ± 0.03	4.22 ± 0.02	4.22
	$V_{tr} = 16K$	3.58 ± 0.01	4.33 ± 0.02	4.02 ± 0.00	3.98
BCL	$V_{tr} = 15.8K$	3.60 ± 0.01	4.32 ± 0.01	4.06 ± 0.01	3.99
VocabTrim-T	$V_{tr} = 64K$	3.93 ± 0.02	4.88 ± 0.03	4.37 ± 0.02	4.39
	$V_{tr} = 32K$	3.82 ± 0.02	4.60 ± 0.01	4.24 ± 0.01	4.22
	$V_{tr} = 16K$	3.59 ± 0.02	4.29 ± 0.02	4.03 ± 0.02	3.97
SpecVocab	$r = d/4$	3.94 ± 0.02	4.93 ± 0.01	4.43 ± 0.02	4.43
	$r = d/8$	3.92 ± 0.02	4.93 ± 0.02	4.45 ± 0.01	4.43
	$r = d/16$	3.96 ± 0.01	4.93 ± 0.01	4.43 ± 0.02	4.44
SlimSpec	$r = d/4$	3.87 ± 0.01	4.89 ± 0.02	4.39 ± 0.02	4.38
	$r = d/8$	3.81 ± 0.02	4.89 ± 0.02	4.39 ± 0.02	4.36
	$r = d/16$	3.78 ± 0.02	4.83 ± 0.01	4.36 ± 0.01	4.32

Table 8: Speedup for Llama3.1-8B-Instruct, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.83 ± 0.02	2.33 ± 0.03	2.02 ± 0.03	2.06
	$V_{tr} = 64K$	1.88 ± 0.03	2.34 ± 0.03	2.00 ± 0.02	2.07
FR-Spec	$V_{tr} = 32K$	1.75 ± 0.03	2.09 ± 0.05	1.85 ± 0.01	1.90
	$V_{tr} = 16K$	1.56 ± 0.01	1.84 ± 0.01	1.70 ± 0.04	1.70
VocabTrim	$V_{tr} = 64K$	1.91 ± 0.03	2.38 ± 0.05	2.05 ± 0.05	2.11
	$V_{tr} = 32K$	1.83 ± 0.05	2.29 ± 0.02	2.01 ± 0.02	2.04
	$V_{tr} = 16K$	1.78 ± 0.02	2.14 ± 0.03	1.94 ± 0.02	1.95
BCL	$V_{tr} = 15.8K$	1.76 ± 0.04	2.18 ± 0.02	2.02 ± 0.04	1.99
	$V_{tr} = 64K$	1.93 ± 0.04	2.39 ± 0.04	2.11 ± 0.01	2.14
VocabTrim-T	$V_{tr} = 32K$	1.92 ± 0.03	2.29 ± 0.03	2.01 ± 0.02	2.08
	$V_{tr} = 16K$	1.80 ± 0.05	2.10 ± 0.05	1.94 ± 0.03	1.95
SpecVocab	$r = d/4$	2.10 ± 0.03	2.62 ± 0.03	2.24 ± 0.03	2.32
	$r = d/8$	2.11 ± 0.05	2.66 ± 0.05	2.32 ± 0.02	2.36
	$r = d/16$	2.17 ± 0.04	2.67 ± 0.03	2.30 ± 0.02	2.38
SlimSpec	$r = d/4$	2.16 ± 0.02	2.68 ± 0.06	2.35 ± 0.01	2.40
	$r = d/8$	2.15 ± 0.02	2.71 ± 0.03	2.31 ± 0.06	2.39
	$r = d/16$	2.13 ± 0.02	2.70 ± 0.03	2.30 ± 0.06	2.37

Table 9: Average acceptance length τ for Llama3.1-8B-Instruct, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.49 ± 0.03	4.46 ± 0.04	3.89 ± 0.03	3.95
	$V_{tr} = 64K$	3.41 ± 0.01	4.33 ± 0.04	3.86 ± 0.04	3.87
FR-Spec	$V_{tr} = 32K$	3.17 ± 0.03	3.85 ± 0.04	3.50 ± 0.03	3.51
	$V_{tr} = 16K$	2.87 ± 0.01	3.43 ± 0.02	3.25 ± 0.02	3.18
VocabTrim	$V_{tr} = 64K$	3.49 ± 0.02	4.39 ± 0.04	3.88 ± 0.04	3.92
	$V_{tr} = 32K$	3.42 ± 0.03	4.24 ± 0.01	3.78 ± 0.05	3.81
	$V_{tr} = 16K$	3.28 ± 0.02	3.95 ± 0.03	3.64 ± 0.02	3.62
BCL	$V_{tr} = 15.8K$	3.28 ± 0.03	3.91 ± 0.05	3.67 ± 0.03	3.62
	$V_{tr} = 64K$	3.49 ± 0.03	4.39 ± 0.02	3.90 ± 0.02	3.93
VocabTrim-T	$V_{tr} = 32K$	3.40 ± 0.05	4.20 ± 0.02	3.74 ± 0.03	3.78
	$V_{tr} = 16K$	3.26 ± 0.03	3.92 ± 0.04	3.63 ± 0.02	3.60
SpecVocab	$r = d/4$	3.51 ± 0.05	4.45 ± 0.03	3.93 ± 0.03	3.96
	$r = d/8$	3.50 ± 0.05	4.45 ± 0.04	3.93 ± 0.04	3.96
	$r = d/16$	3.53 ± 0.04	4.43 ± 0.05	3.92 ± 0.03	3.96
SlimSpec	$r = d/4$	3.46 ± 0.04	4.41 ± 0.06	3.89 ± 0.01	3.92
	$r = d/8$	3.42 ± 0.03	4.42 ± 0.05	3.84 ± 0.05	3.89
	$r = d/16$	3.34 ± 0.02	4.34 ± 0.05	3.82 ± 0.03	3.83

Table 10: Speedup for Llama3.1-8B-Instruct, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.19 ± 0.03	1.28 ± 0.05	1.28 ± 0.03	1.25
FR-Spec	$V_{tr} = 64K$	1.21 ± 0.00	1.32 ± 0.03	1.32 ± 0.01	1.28
	$V_{tr} = 32K$	1.16 ± 0.02	1.30 ± 0.03	1.26 ± 0.03	1.24
	$V_{tr} = 16K$	1.08 ± 0.03	1.15 ± 0.04	1.21 ± 0.02	1.15
VocabTrim	$V_{tr} = 64K$	1.22 ± 0.03	1.36 ± 0.04	1.31 ± 0.03	1.30
	$V_{tr} = 32K$	1.23 ± 0.05	1.32 ± 0.01	1.28 ± 0.04	1.27
	$V_{tr} = 16K$	1.18 ± 0.02	1.31 ± 0.04	1.31 ± 0.04	1.27
BCL	$V_{tr} = 15.8K$	1.21 ± 0.05	1.29 ± 0.03	1.33 ± 0.04	1.28
VocabTrim-T	$V_{tr} = 64K$	1.23 ± 0.03	1.35 ± 0.04	1.30 ± 0.03	1.29
	$V_{tr} = 32K$	1.19 ± 0.04	1.33 ± 0.02	1.30 ± 0.02	1.27
	$V_{tr} = 16K$	1.16 ± 0.03	1.30 ± 0.03	1.30 ± 0.02	1.25
SpecVocab	$r = d/4$	1.22 ± 0.05	1.24 ± 0.20	1.29 ± 0.06	1.25
	$r = d/8$	1.17 ± 0.04	1.30 ± 0.05	1.32 ± 0.05	1.26
	$r = d/16$	1.17 ± 0.04	1.19 ± 0.27	1.34 ± 0.03	1.24
SlimSpec	$r = d/4$	1.24 ± 0.05	1.42 ± 0.05	1.36 ± 0.06	1.34
	$r = d/8$	1.23 ± 0.06	1.42 ± 0.04	1.38 ± 0.05	1.35
	$r = d/16$	1.27 ± 0.03	1.36 ± 0.04	1.36 ± 0.05	1.33

Table 11: Average acceptance length τ for Llama3.1-8B-Instruct, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.47 ± 0.03	4.43 ± 0.05	3.86 ± 0.03	3.92
FR-Spec	$V_{tr} = 64K$	3.38 ± 0.04	4.29 ± 0.04	3.80 ± 0.04	3.82
	$V_{tr} = 32K$	3.13 ± 0.04	3.89 ± 0.04	3.45 ± 0.03	3.49
	$V_{tr} = 16K$	2.86 ± 0.03	3.40 ± 0.04	3.25 ± 0.04	3.17
VocabTrim	$V_{tr} = 64K$	3.46 ± 0.03	4.37 ± 0.04	3.81 ± 0.05	3.88
	$V_{tr} = 32K$	3.41 ± 0.02	4.21 ± 0.04	3.79 ± 0.03	3.80
	$V_{tr} = 16K$	3.22 ± 0.04	3.95 ± 0.02	3.58 ± 0.02	3.58
BCL	$V_{tr} = 15.8K$	3.25 ± 0.03	3.93 ± 0.03	3.66 ± 0.04	3.61
VocabTrim-T	$V_{tr} = 64K$	3.48 ± 0.05	4.39 ± 0.07	3.83 ± 0.05	3.90
	$V_{tr} = 32K$	3.41 ± 0.02	4.19 ± 0.03	3.74 ± 0.06	3.78
	$V_{tr} = 16K$	3.23 ± 0.00	3.92 ± 0.04	3.60 ± 0.04	3.58
SpecVocab	$r = d/4$	3.51 ± 0.04	4.43 ± 0.03	3.87 ± 0.04	3.94
	$r = d/8$	3.52 ± 0.05	4.44 ± 0.02	3.94 ± 0.03	3.97
	$r = d/16$	3.51 ± 0.02	4.45 ± 0.05	3.90 ± 0.05	3.95
SlimSpec	$r = d/4$	3.46 ± 0.03	4.45 ± 0.03	3.91 ± 0.05	3.94
	$r = d/8$	3.41 ± 0.02	4.47 ± 0.05	3.88 ± 0.04	3.92
	$r = d/16$	3.35 ± 0.02	4.32 ± 0.05	3.84 ± 0.05	3.84

Table 12: Speedup for GPT-OSS-20B, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.43 ± 0.00	1.41 ± 0.00	1.63 ± 0.01	1.49
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.64 ± 0.01	1.57 ± 0.01	1.82 ± 0.00	1.68
	$V_{\text{tr}} = 32\text{K}$	1.52 ± 0.01	1.50 ± 0.01	1.71 ± 0.02	1.57
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.71 ± 0.01	1.66 ± 0.01	1.89 ± 0.01	1.75
	$V_{\text{tr}} = 32\text{K}$	1.64 ± 0.01	1.60 ± 0.01	1.87 ± 0.01	1.71
BCL	$V_{\text{tr}} = 81.7\text{K}$	1.72 ± 0.02	1.78 ± 0.02	1.99 ± 0.01	1.83
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	1.72 ± 0.01	1.67 ± 0.00	1.95 ± 0.00	1.78
	$V_{\text{tr}} = 32\text{K}$	1.76 ± 0.00	1.71 ± 0.01	1.96 ± 0.00	1.81
SpecVocab	$r = d/8$	1.74 ± 0.00	1.71 ± 0.01	1.98 ± 0.01	1.81
SlimSpec	$r = d/4$	1.77 ± 0.01	1.75 ± 0.01	2.06 ± 0.01	1.86
	$r = d/8$	1.76 ± 0.02	1.76 ± 0.01	2.11 ± 0.01	1.88
	$r = d/16$	1.76 ± 0.01	1.74 ± 0.01	2.05 ± 0.00	1.85

Table 13: Average acceptance length τ for GPT-OSS-20B, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.30 ± 0.03	3.36 ± 0.00	3.89 ± 0.02	3.52
FR-Spec	$V_{\text{tr}} = 64\text{K}$	3.15 ± 0.02	3.10 ± 0.01	3.62 ± 0.01	3.29
	$V_{\text{tr}} = 32\text{K}$	2.92 ± 0.03	2.95 ± 0.00	3.38 ± 0.00	3.08
VocabTrim	$V_{\text{tr}} = 64\text{K}$	3.27 ± 0.00	3.25 ± 0.01	3.71 ± 0.01	3.41
	$V_{\text{tr}} = 32\text{K}$	3.16 ± 0.01	3.12 ± 0.01	3.63 ± 0.01	3.30
BCL	$V_{\text{tr}} = 81.7\text{K}$	3.34 ± 0.03	3.41 ± 0.01	3.95 ± 0.00	3.57
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	3.34 ± 0.00	3.35 ± 0.00	3.95 ± 0.01	3.55
	$V_{\text{tr}} = 32\text{K}$	3.20 ± 0.01	3.19 ± 0.02	3.69 ± 0.01	3.36
SpecVocab	$r = d/8$	3.37 ± 0.00	3.40 ± 0.02	3.99 ± 0.01	3.59
SlimSpec	$r = d/4$	3.34 ± 0.01	3.39 ± 0.02	4.03 ± 0.00	3.59
	$r = d/8$	3.18 ± 0.02	3.30 ± 0.01	3.99 ± 0.01	3.49
	$r = d/16$	3.20 ± 0.01	3.25 ± 0.02	3.90 ± 0.00	3.45

Table 14: Speedup for GPT-OSS-20B, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.36 ± 0.03	1.17 ± 0.02	1.38 ± 0.01	1.30
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.38 ± 0.03	1.15 ± 0.03	1.42 ± 0.00	1.32
	$V_{\text{tr}} = 32\text{K}$	1.35 ± 0.01	1.15 ± 0.02	1.40 ± 0.02	1.30
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.52 ± 0.01	1.29 ± 0.03	1.48 ± 0.02	1.43
	$V_{\text{tr}} = 32\text{K}$	1.50 ± 0.03	1.27 ± 0.00	1.49 ± 0.00	1.42
BCL	$V_{\text{tr}} = 81.7\text{K}$	1.49 ± 0.02	1.34 ± 0.04	1.56 ± 0.03	1.46
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	1.50 ± 0.04	1.27 ± 0.01	1.52 ± 0.03	1.43
	$V_{\text{tr}} = 32\text{K}$	1.41 ± 0.02	1.20 ± 0.02	1.44 ± 0.06	1.35
SpecVocab	$r = d/8$	1.43 ± 0.02	1.18 ± 0.03	1.45 ± 0.02	1.35
SlimSpec	$r = d/4$	1.44 ± 0.01	1.23 ± 0.06	1.50 ± 0.02	1.39
	$r = d/8$	1.52 ± 0.03	1.34 ± 0.02	1.56 ± 0.04	1.47
	$r = d/16$	1.45 ± 0.03	1.20 ± 0.04	1.48 ± 0.05	1.38

Table 15: Average acceptance length τ for GPT-OSS-20B, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.33 ± 0.03	3.36 ± 0.02	3.91 ± 0.02	3.53
FR-Spec	$V_{\text{tr}} = 64\text{K}$	3.10 ± 0.04	3.10 ± 0.01	3.62 ± 0.01	3.27
	$V_{\text{tr}} = 32\text{K}$	2.92 ± 0.02	2.95 ± 0.01	3.39 ± 0.00	3.09
VocabTrim	$V_{\text{tr}} = 64\text{K}$	3.26 ± 0.02	3.26 ± 0.02	3.72 ± 0.01	3.41
	$V_{\text{tr}} = 32\text{K}$	3.13 ± 0.01	3.12 ± 0.02	3.61 ± 0.02	3.29
BCL	$V_{\text{tr}} = 81.7\text{K}$	3.38 ± 0.02	3.39 ± 0.01	3.94 ± 0.02	3.57
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	3.32 ± 0.01	3.35 ± 0.02	3.96 ± 0.02	3.54
	$V_{\text{tr}} = 32\text{K}$	3.19 ± 0.01	3.20 ± 0.02	3.69 ± 0.02	3.36
SpecVocab	$r = d/8$	3.37 ± 0.03	3.39 ± 0.02	3.99 ± 0.02	3.58
SlimSpec	$r = d/4$	3.38 ± 0.02	3.38 ± 0.01	4.03 ± 0.02	3.60
	$r = d/8$	3.29 ± 0.02	3.34 ± 0.02	3.95 ± 0.03	3.53
	$r = d/16$	3.21 ± 0.02	3.24 ± 0.02	3.89 ± 0.01	3.45

Table 16: Speedup for GPT-OSS-20B, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.24 ± 0.01	1.28 ± 0.01	1.47 ± 0.01	1.33
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.17 ± 0.01	1.23 ± 0.02	1.38 ± 0.05	1.26
	$V_{\text{tr}} = 32\text{K}$	1.07 ± 0.01	1.17 ± 0.01	1.28 ± 0.01	1.17
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.21 ± 0.04	1.33 ± 0.02	1.42 ± 0.02	1.32
	$V_{\text{tr}} = 32\text{K}$	1.19 ± 0.01	1.20 ± 0.02	1.36 ± 0.04	1.25
BCL	$V_{\text{tr}} = 81.7\text{K}$	1.29 ± 0.03	1.35 ± 0.03	1.57 ± 0.02	1.40
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	1.28 ± 0.01	1.25 ± 0.02	1.48 ± 0.04	1.34
	$V_{\text{tr}} = 32\text{K}$	1.38 ± 0.01	1.43 ± 0.01	1.68 ± 0.02	1.49
SpecVocab	$r = d/8$	1.39 ± 0.01	1.46 ± 0.02	1.71 ± 0.01	1.52
SlimSpec	$r = d/4$	1.43 ± 0.02	1.52 ± 0.01	1.74 ± 0.01	1.56
	$r = d/8$	1.40 ± 0.02	1.47 ± 0.03	1.72 ± 0.02	1.53
	$r = d/16$	1.32 ± 0.01	1.38 ± 0.01	1.64 ± 0.02	1.44

Table 17: Average acceptance length τ for GPT-OSS-20B, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	2.96 ± 0.02	3.08 ± 0.02	3.44 ± 0.03	3.16
FR-Spec	$V_{\text{tr}} = 64\text{K}$	2.78 ± 0.03	2.89 ± 0.01	3.22 ± 0.02	2.96
	$V_{\text{tr}} = 32\text{K}$	2.62 ± 0.02	2.76 ± 0.02	3.04 ± 0.01	2.81
VocabTrim	$V_{\text{tr}} = 64\text{K}$	2.89 ± 0.02	2.99 ± 0.03	3.30 ± 0.02	3.06
	$V_{\text{tr}} = 32\text{K}$	2.80 ± 0.01	2.88 ± 0.02	3.21 ± 0.02	2.96
BCL	$V_{\text{tr}} = 81.7\text{K}$	3.08 ± 0.01	3.22 ± 0.02	3.60 ± 0.03	3.30
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	2.95 ± 0.03	3.08 ± 0.04	3.47 ± 0.02	3.17
	$V_{\text{tr}} = 32\text{K}$	2.87 ± 0.03	2.99 ± 0.01	3.32 ± 0.01	3.06
SpecVocab	$r = d/8$	2.93 ± 0.02	3.13 ± 0.02	3.50 ± 0.01	3.19
SlimSpec	$r = d/4$	2.96 ± 0.02	3.16 ± 0.02	3.54 ± 0.02	3.22
	$r = d/8$	2.83 ± 0.03	3.02 ± 0.02	3.42 ± 0.02	3.09
	$r = d/16$	2.76 ± 0.03	2.92 ± 0.01	3.35 ± 0.01	3.01

Table 18: Speedup for GPT-OSS-20B, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.25 ± 0.02	1.17 ± 0.01	1.28 ± 0.01	1.23
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.31 ± 0.02	1.15 ± 0.03	1.30 ± 0.02	1.25
	$V_{\text{tr}} = 32\text{K}$	1.26 ± 0.04	1.14 ± 0.01	1.26 ± 0.02	1.22
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.30 ± 0.04	1.24 ± 0.02	1.33 ± 0.02	1.29
	$V_{\text{tr}} = 32\text{K}$	1.28 ± 0.05	1.25 ± 0.03	1.32 ± 0.02	1.28
BCL	$V_{\text{tr}} = 81.7\text{K}$	1.30 ± 0.04	1.28 ± 0.02	1.38 ± 0.03	1.32
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	1.34 ± 0.04	1.26 ± 0.04	1.36 ± 0.02	1.32
	$V_{\text{tr}} = 32\text{K}$	1.29 ± 0.04	1.15 ± 0.04	1.36 ± 0.02	1.27
SpecVocab	$r = d/8$	1.29 ± 0.02	1.18 ± 0.05	1.30 ± 0.08	1.26
SlimSpec	$r = d/4$	1.34 ± 0.03	1.20 ± 0.02	1.36 ± 0.07	1.30
	$r = d/8$	1.32 ± 0.03	1.28 ± 0.02	1.35 ± 0.06	1.32
	$r = d/16$	1.17 ± 0.26	1.17 ± 0.02	1.33 ± 0.06	1.22

Table 19: Average acceptance length τ for GPT-OSS-20B, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	2.95 ± 0.02	3.08 ± 0.03	3.41 ± 0.02	3.15
FR-Spec	$V_{\text{tr}} = 64\text{K}$	2.80 ± 0.02	2.88 ± 0.03	3.22 ± 0.02	2.97
	$V_{\text{tr}} = 32\text{K}$	2.62 ± 0.01	2.75 ± 0.02	3.03 ± 0.02	2.80
VocabTrim	$V_{\text{tr}} = 64\text{K}$	2.86 ± 0.01	3.00 ± 0.02	3.29 ± 0.01	3.05
	$V_{\text{tr}} = 32\text{K}$	2.79 ± 0.03	2.91 ± 0.02	3.20 ± 0.03	2.97
BCL	$V_{\text{tr}} = 81.7\text{K}$	3.10 ± 0.03	3.23 ± 0.00	3.63 ± 0.02	3.32
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	2.94 ± 0.01	3.06 ± 0.03	3.47 ± 0.02	3.16
	$V_{\text{tr}} = 32\text{K}$	2.86 ± 0.02	2.99 ± 0.03	3.32 ± 0.02	3.06
SpecVocab	$r = d/8$	2.95 ± 0.03	3.13 ± 0.02	3.49 ± 0.03	3.19
SlimSpec	$r = d/4$	2.96 ± 0.02	3.15 ± 0.02	3.55 ± 0.01	3.22
	$r = d/8$	2.84 ± 0.03	3.02 ± 0.05	3.40 ± 0.03	3.09
	$r = d/16$	2.75 ± 0.02	2.92 ± 0.02	3.35 ± 0.01	3.01

Table 20: Speedup for Qwen3-30B-A3B, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.55 ± 0.02	2.07 ± 0.01	2.12 ± 0.01	1.91
FR-Spec	$V_{tr} = 64K$	1.59 ± 0.01	2.15 ± 0.02	2.13 ± 0.01	1.96
	$V_{tr} = 32K$	1.51 ± 0.00	1.95 ± 0.01	1.92 ± 0.01	1.79
VocabTrim	$V_{tr} = 64K$	1.65 ± 0.01	2.19 ± 0.02	2.22 ± 0.01	2.02
	$V_{tr} = 32K$	1.66 ± 0.01	2.18 ± 0.01	2.22 ± 0.02	2.02
BCL	$V_{tr} = 66.9K$	1.62 ± 0.00	2.08 ± 0.01	2.16 ± 0.01	1.95
VocabTrim-T	$V_{tr} = 64K$	1.66 ± 0.01	2.15 ± 0.01	2.23 ± 0.01	2.01
	$V_{tr} = 32K$	1.66 ± 0.01	2.15 ± 0.02	2.22 ± 0.01	2.01
SpecVocab	$r = d/8$	1.66 ± 0.01	2.19 ± 0.01	2.24 ± 0.01	2.03
SlimSpec	$r = d/4$	1.68 ± 0.03	2.22 ± 0.03	2.27 ± 0.02	2.05
	$r = d/8$	1.66 ± 0.00	2.22 ± 0.02	2.27 ± 0.02	2.05
	$r = d/16$	1.61 ± 0.00	2.19 ± 0.01	2.23 ± 0.00	2.01

Table 21: Average acceptance length τ for Qwen3-30B-A3B, temperature = 0, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.12 ± 0.00	4.37 ± 0.01	4.52 ± 0.00	4.00
FR-Spec	$V_{tr} = 64K$	2.99 ± 0.01	4.26 ± 0.01	4.32 ± 0.01	3.86
	$V_{tr} = 32K$	2.76 ± 0.01	3.78 ± 0.00	3.78 ± 0.01	3.44
VocabTrim	$V_{tr} = 64K$	3.11 ± 0.00	4.35 ± 0.00	4.51 ± 0.00	3.99
	$V_{tr} = 32K$	3.06 ± 0.00	4.22 ± 0.01	4.40 ± 0.00	3.89
BCL	$V_{tr} = 66.9K$	2.96 ± 0.00	3.98 ± 0.01	4.25 ± 0.00	3.73
VocabTrim-T	$V_{tr} = 64K$	3.12 ± 0.00	4.29 ± 0.01	4.52 ± 0.00	3.98
	$V_{tr} = 32K$	3.05 ± 0.00	4.17 ± 0.00	4.40 ± 0.00	3.87
SpecVocab	$r = d/8$	3.13 ± 0.00	4.36 ± 0.01	4.54 ± 0.01	4.01
SlimSpec	$r = d/4$	3.09 ± 0.00	4.31 ± 0.00	4.52 ± 0.01	3.97
	$r = d/8$	3.00 ± 0.01	4.26 ± 0.00	4.47 ± 0.00	3.91
	$r = d/16$	2.92 ± 0.01	4.20 ± 0.00	4.37 ± 0.00	3.83

Table 22: Speedup for Qwen3-30B-A3B, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.22 ± 0.02	1.32 ± 0.02	1.48 ± 0.02	1.34
FR-Spec	$V_{\text{tr}} = 64\text{K}$	1.27 ± 0.01	1.36 ± 0.03	1.48 ± 0.02	1.37
	$V_{\text{tr}} = 32\text{K}$	1.24 ± 0.02	1.31 ± 0.03	1.40 ± 0.02	1.32
VocabTrim	$V_{\text{tr}} = 64\text{K}$	1.30 ± 0.02	1.36 ± 0.02	1.51 ± 0.02	1.39
	$V_{\text{tr}} = 32\text{K}$	1.29 ± 0.02	1.35 ± 0.03	1.52 ± 0.02	1.39
BCL	$V_{\text{tr}} = 66.9\text{K}$	1.32 ± 0.03	1.35 ± 0.01	1.49 ± 0.03	1.39
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	1.31 ± 0.02	1.36 ± 0.02	1.53 ± 0.04	1.40
	$V_{\text{tr}} = 32\text{K}$	1.30 ± 0.02	1.35 ± 0.02	1.50 ± 0.01	1.38
SpecVocab	$r = d/8$	1.28 ± 0.03	1.34 ± 0.11	1.49 ± 0.02	1.37
SlimSpec	$r = d/4$	1.31 ± 0.04	1.37 ± 0.05	1.52 ± 0.04	1.40
	$r = d/8$	1.31 ± 0.02	1.36 ± 0.03	1.54 ± 0.03	1.40
	$r = d/16$	1.31 ± 0.04	1.37 ± 0.02	1.49 ± 0.04	1.39

Table 23: Average acceptance length τ for Qwen3-30B-A3B, temperature = 0, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	3.12 ± 0.01	4.30 ± 0.04	4.50 ± 0.03	3.97
FR-Spec	$V_{\text{tr}} = 64\text{K}$	2.99 ± 0.01	4.17 ± 0.05	4.29 ± 0.01	3.82
	$V_{\text{tr}} = 32\text{K}$	2.77 ± 0.01	3.69 ± 0.03	3.74 ± 0.02	3.40
VocabTrim	$V_{\text{tr}} = 64\text{K}$	3.12 ± 0.01	4.25 ± 0.04	4.47 ± 0.02	3.95
	$V_{\text{tr}} = 32\text{K}$	3.07 ± 0.01	4.13 ± 0.03	4.37 ± 0.03	3.86
BCL	$V_{\text{tr}} = 66.9\text{K}$	2.93 ± 0.01	3.92 ± 0.01	4.26 ± 0.01	3.70
VocabTrim-T	$V_{\text{tr}} = 64\text{K}$	3.13 ± 0.01	4.24 ± 0.02	4.51 ± 0.01	3.96
	$V_{\text{tr}} = 32\text{K}$	3.06 ± 0.01	4.12 ± 0.02	4.40 ± 0.01	3.86
SpecVocab	$r = d/8$	3.14 ± 0.01	4.34 ± 0.02	4.56 ± 0.01	4.01
SlimSpec	$r = d/4$	3.07 ± 0.01	4.26 ± 0.04	4.50 ± 0.00	3.94
	$r = d/8$	3.01 ± 0.01	4.22 ± 0.02	4.45 ± 0.02	3.89
	$r = d/16$	2.92 ± 0.01	4.17 ± 0.02	4.36 ± 0.01	3.82

Table 24: Speedup for Qwen3-30B-A3B, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.18 ± 0.01	1.58 ± 0.01	1.63 ± 0.01	1.46
FR-Spec	$V_{tr} = 64K$	1.18 ± 0.01	1.62 ± 0.01	1.63 ± 0.02	1.48
	$V_{tr} = 32K$	1.13 ± 0.01	1.50 ± 0.01	1.49 ± 0.01	1.37
VocabTrim	$V_{tr} = 64K$	1.22 ± 0.02	1.67 ± 0.01	1.69 ± 0.01	1.53
	$V_{tr} = 32K$	1.22 ± 0.02	1.66 ± 0.01	1.69 ± 0.02	1.52
BCL	$V_{tr} = 66.9K$	1.19 ± 0.02	1.57 ± 0.01	1.65 ± 0.02	1.47
VocabTrim-T	$V_{tr} = 64K$	1.22 ± 0.01	1.65 ± 0.02	1.70 ± 0.01	1.52
	$V_{tr} = 32K$	1.21 ± 0.02	1.63 ± 0.02	1.69 ± 0.02	1.51
SpecVocab	$r = d/8$	1.26 ± 0.01	1.67 ± 0.02	1.72 ± 0.01	1.55
SlimSpec	$r = d/4$	1.25 ± 0.01	1.73 ± 0.04	1.76 ± 0.02	1.58
	$r = d/8$	1.26 ± 0.01	1.72 ± 0.02	1.77 ± 0.01	1.58
	$r = d/16$	1.21 ± 0.01	1.71 ± 0.01	1.73 ± 0.01	1.55

Table 25: Average acceptance length τ for Qwen3-30B-A3B, temperature = 1, batch size = 1.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	2.69 ± 0.03	3.76 ± 0.04	3.94 ± 0.03	3.46
FR-Spec	$V_{tr} = 64K$	2.60 ± 0.02	3.69 ± 0.02	3.80 ± 0.03	3.36
	$V_{tr} = 32K$	2.44 ± 0.02	3.34 ± 0.02	3.39 ± 0.01	3.06
VocabTrim	$V_{tr} = 64K$	2.70 ± 0.02	3.75 ± 0.03	3.93 ± 0.02	3.46
	$V_{tr} = 32K$	2.66 ± 0.03	3.70 ± 0.03	3.86 ± 0.03	3.41
BCL	$V_{tr} = 66.9K$	2.55 ± 0.02	3.48 ± 0.03	3.76 ± 0.01	3.26
VocabTrim-T	$V_{tr} = 64K$	2.67 ± 0.02	3.72 ± 0.05	3.92 ± 0.02	3.44
	$V_{tr} = 32K$	2.64 ± 0.02	3.62 ± 0.04	3.86 ± 0.01	3.37
SpecVocab	$r = d/8$	2.69 ± 0.01	3.76 ± 0.04	3.97 ± 0.03	3.47
SlimSpec	$r = d/4$	2.65 ± 0.02	3.83 ± 0.05	3.99 ± 0.02	3.49
	$r = d/8$	2.61 ± 0.02	3.75 ± 0.03	3.96 ± 0.02	3.44
	$r = d/16$	2.53 ± 0.02	3.71 ± 0.03	3.87 ± 0.01	3.37

Table 26: Speedup for Qwen3-30B-A3B, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	1.11 ± 0.03	1.21 ± 0.04	1.32 ± 0.01	1.21
FR-Spec	$V_{tr} = 64K$	1.19 ± 0.02	1.23 ± 0.04	1.36 ± 0.01	1.26
	$V_{tr} = 32K$	1.11 ± 0.07	1.20 ± 0.05	1.28 ± 0.03	1.20
VocabTrim	$V_{tr} = 64K$	1.19 ± 0.02	1.23 ± 0.03	1.37 ± 0.03	1.26
	$V_{tr} = 32K$	1.18 ± 0.01	1.27 ± 0.04	1.36 ± 0.02	1.27
BCL	$V_{tr} = 66.9K$	1.20 ± 0.02	1.26 ± 0.02	1.35 ± 0.02	1.27
VocabTrim-T	$V_{tr} = 64K$	1.19 ± 0.03	1.27 ± 0.04	1.38 ± 0.01	1.28
	$V_{tr} = 32K$	1.18 ± 0.02	1.26 ± 0.02	1.35 ± 0.05	1.26
SpecVocab	$r = d/8$	1.09 ± 0.17	1.24 ± 0.04	1.32 ± 0.10	1.21
SlimSpec	$r = d/4$	1.18 ± 0.04	1.19 ± 0.20	1.35 ± 0.10	1.24
	$r = d/8$	1.19 ± 0.02	1.28 ± 0.02	1.39 ± 0.04	1.29
	$r = d/16$	1.18 ± 0.03	1.29 ± 0.02	1.34 ± 0.09	1.27

Table 27: Average acceptance length τ for Qwen3-30B-A3B, temperature = 1, batch size = 64.

Method	Config	MT-Bench	Humaneval	GSM8K	Avg
Full Vocab	–	2.66 ± 0.00	3.70 ± 0.07	3.88 ± 0.04	3.41
FR-Spec	$V_{tr} = 64K$	2.58 ± 0.02	3.60 ± 0.06	3.72 ± 0.06	3.30
	$V_{tr} = 32K$	2.43 ± 0.02	3.27 ± 0.02	3.37 ± 0.03	3.02
VocabTrim	$V_{tr} = 64K$	2.66 ± 0.03	3.67 ± 0.08	3.87 ± 0.03	3.40
	$V_{tr} = 32K$	2.63 ± 0.02	3.61 ± 0.07	3.81 ± 0.05	3.35
BCL	$V_{tr} = 66.9K$	2.57 ± 0.02	3.43 ± 0.04	3.75 ± 0.02	3.25
VocabTrim-T	$V_{tr} = 64K$	2.67 ± 0.02	3.69 ± 0.03	3.92 ± 0.02	3.43
	$V_{tr} = 32K$	2.62 ± 0.02	3.61 ± 0.03	3.79 ± 0.01	3.34
SpecVocab	$r = d/8$	2.67 ± 0.02	3.74 ± 0.02	3.95 ± 0.04	3.45
SlimSpec	$r = d/4$	2.67 ± 0.03	3.82 ± 0.04	3.97 ± 0.03	3.49
	$r = d/8$	2.59 ± 0.01	3.72 ± 0.01	3.94 ± 0.02	3.42
	$r = d/16$	2.52 ± 0.02	3.68 ± 0.03	3.85 ± 0.03	3.35